

Prototyp Skizze „Krakensuche“

Beschreibung

„Krakensuche“ ist eine verteilte Mini-Suchmaschine. Ein Master-Knoten koordiniert mehrere Crawler-Worker, die parallel Webseiten herunterladen, Text und Links extrahieren und ihre Ergebnisse an den Master zurückmelden. Der Master baut daraus einen invertierten Index auf, über den Nutzer anschließend per Volltextsuche nach Relevanz sortierte Ergebnisse abrufen können. Das System motiviert Verteilung und Nebenläufigkeit über ihr Kern-NFA: Skalierbarkeit und Durchsatz während des Crawls.

Funktionale Anforderungen

- Nutzer können Crawl-Jobs mit Seed-URLs, maximaler Tiefe, maximaler Seitenzahl und Domain-Whitelist anlegen, pausieren, fortsetzen, abbrechen und löschen
- Nutzer können den Status eines Crawl-Jobs abrufen (gecrawlte Seiten, Warteschlangen-Tiefe, aktive Worker, Fehlerzähler)
- Nutzer können Volltextsuchen mit einem oder mehreren Schlüsselwörtern ausführen und erhalten ein nach TF-IDF-Relevanz sortiertes Ergebnis mit Snippet und URL
- Nutzer können einzelne Dokumente per ID abrufen, um Metadaten und extrahierten Text einzusehen
- Nutzer können Statistiken abrufen (Anzahl Dokumente, Index-Größe, Crawl-Durchsatz, aktive Worker, durchschnittliche Antwortzeiten)
- Der Master verteilt zu crawlende URLs über Sockets an registrierte Worker und empfängt die gecrawlten Ergebnisse (Text, gefundene Links, HTTP-Status) zurück
- Worker registrieren und deregistrieren sich zur Laufzeit beim Master über eine persistente Socket-Verbindung mit Heartbeat
- Der Master dedupliziert URLs je Job (dieselbe URL wird innerhalb eines Jobs nur einmal gecrawlt)
- Der Master aktualisiert den invertierten Index inkrementell, sobald neue Dokumente eintreffen
- Das System erstellt Logs zu allen relevanten Ereignissen (Crawl-Starts, Worker-Registrierungen, Fehler, Suchanfragen)

Nicht-funktionale Anforderungen

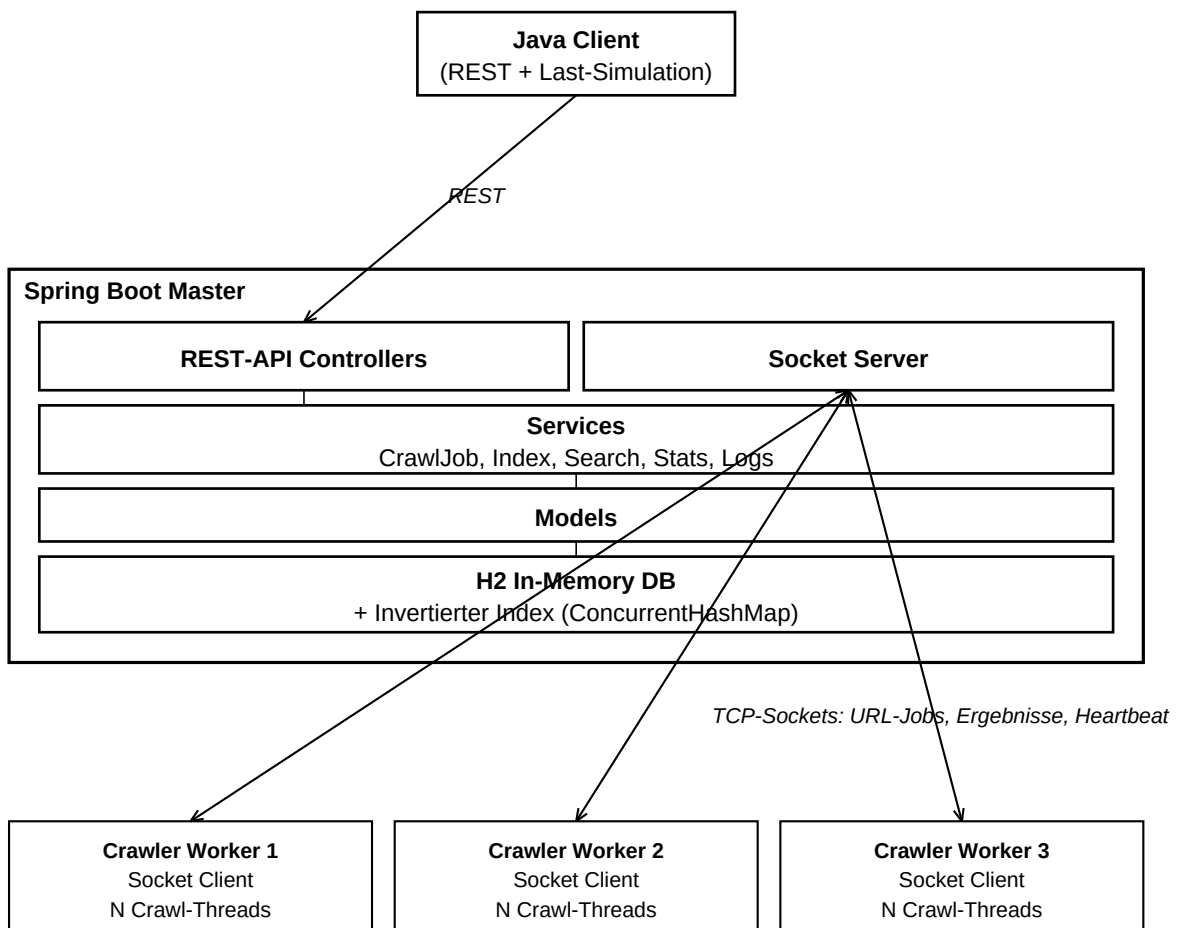
Bei diesen Anforderungen wird davon ausgegangen, dass Master und Worker auf einem handelsüblichen Laptop (mehrkernige CPU) laufen.

- Der Master beantwortet korrekte REST-Anfragen zu >99,5% ohne internen Fehler
- Suchanfragen auf einem Index mit bis zu 10.000 Dokumenten werden in unter 200 ms beantwortet (bei einer Rate von ≤ 20 Anfragen/s)
- Der Master kann mit ≥ 3 parallel angebundenen Workern kommunizieren, ohne dass der Koordinations-Overhead den Crawl-Durchsatz limitiert
- Crawler-Worker nutzen Multithreading dynamisch auf mehrkernigen Systemen (N parallele Crawl-Threads, N konfigurierbar bzw. an verfügbare Kerne angepasst)
- URL-Deduplizierung und Updates am invertierten Index sind atomar und thread-safe: keine doppelten Index-Einträge, keine Lost-Updates bei konkurrenten Schreibzugriffen
- Mit 2 Workern soll der Crawl-Durchsatz (Seiten/Sekunde) gegenüber 1 Worker um $\geq 60\%$ steigen (Skalierbarkeits-Nachweis)
- Der Master ist innerhalb von 15 s nach Start bereit, REST-Anfragen und Socket-Verbindungen entgegenzunehmen, wobei das Programm bereits mit allen Dependencies gebaut wurde
- Security wird als einzige nFA explizit ausgenommen, um die Nachvollziehbarkeit und Ausführung des Prototyps einfach zu halten

Verteilung

Nebenläufigkeit	Multithreading pro Worker: paralleles Herunterladen und Parsen mehrerer Seiten (ExecutorService mit konfigurierbarer Pool-Größe) Paralleles Tokenisieren und Indexieren eingehender Dokumente im Master Parallele TF-IDF-Scoring-Berechnung bei Suchanfragen über Dokument-Shards
Verteilung	Master ↔ Worker-Kommunikation via TCP-Sockets (URL-Zuweisung, Ergebnis-Rückmeldung, Heartbeat) Worker als eigenständiger Java-Prozess mit persistenter Socket-Verbindung zum Master (theoretisch auf separatem Rechner lauffähig)
Komponenten	Master als Spring-Boot-Anwendung mit REST-Controllern, Services (CrawlJob, Index, Search, Stats) und eigenem Socket-Server als Spring-Komponente Java-Client für REST (Volltextsuche, Crawl-Steuerung, Stats) und Last-Simulation

Skizze



Last-Simulation & Test der nicht-funktionalen Anforderungen

Nicht-funktionale Anforderung	Überprüfung
>99,5% der korrekten REST-Anfragen ohne internen Fehler	Server-Logging unterscheidet zwischen fehlerhaften Requests und internen Fehlern. Der Client stellt ein breites Spektrum an Anfragen (Jobs, Such-Queries, Dokument-Lookups) und loggt das Verhältnis von Fehlern zu Gesamtanfragen.
Suchanfragen unter 200 ms bei ≤ 20 Anfragen/s auf bis zu 10.000 Dokumenten	Der Performance-Client befüllt den Index zuerst über einen gesteuerten Crawl-Lauf und sendet anschließend getaktet Suchanfragen mit unterschiedlichen Queries. Für jede Anfrage wird die Antwortzeit gemessen und der Anteil über 200 ms ausgegeben.
>60% Durchsatz-Steigerung von 1 auf 2 Worker	Der gleiche fest definierte Crawl-Auftrag (gleiche Seed-URLs, gleiche maximale Seitenzahl) wird einmal mit einem und einmal mit zwei Workern ausgeführt. Der Master loggt Start- und Endzeitpunkt sowie die Anzahl verarbeiteter Dokumente; der Client berechnet den Durchsatz (Seiten/s) und das Verhältnis.
Dynamische Auslastung mehrerer Kerne je Worker	Crawl-Threads loggen Thread-ID und Zeitstempel. Aus dem Log wird die Verteilung der Aufgaben über die Threads berechnet; bei gleichmäßiger Verteilung (Standardabweichung < 20% vom Mittelwert) gilt die NFA als erfüllt.
Atomare, thread-safe Index-Updates	Der Performance-Client füttert den Master parallel mit 1000 identischen Dokument-Indexierungen. Anschließend wird die Posting-Liste eines Test-Terms abgefragt. Ist die Anzahl der Vorkommen ungleich der erwarteten Zahl, ist Atomarität nicht gegeben.
Master in ≤ 15 s startbereit	Überprüfung in den Spring-Logs. Dort ist die Startzeit vermerkt; zusätzlich sendet der Client unmittelbar nach Start einen Health-Check-Request und misst die Zeit bis zur ersten erfolgreichen Antwort.